

Linear Search.

a) Find a given value v in an array A

Input: A sequence of n numbers

A = $\langle a_1, a_2, \dots, a_n \rangle$, and a value v

output: An index i s.t. $v = A[i]$, or the
Special value NIL if v does not appear in A

Linear - Search (A, v)

$i = \text{NIL}$

for $j = 1$ to $A.\text{length}$ do

 if $A[j] == v$ then

$i = j$

 return i

 end if

end for

return i

$T(n) = O(n)$

Linear Search

b) Find minimum element in the given array A

Input: A seq. of n numbers $A = \langle a_1, a_2, \dots, a_n \rangle$.

Output: The index i of the minimum element of A

Find-min (A)

$i = 1$

for $j = 2$ to $A.length$

 if $A[j] < A[i]$

$i = j$

 end if

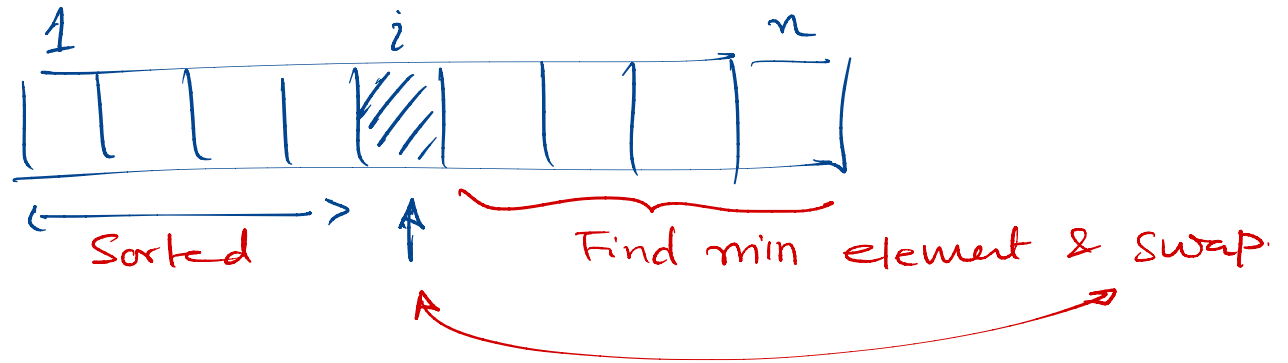
end for

return i

$T(n) = O(n)$

Selection Sort

Sort n numbers by iteratively select the minimum in the subarray and swapping it with the first element of the subarray.



loop invariant: The subarray $A[1:i]$ contains i smallest elements of A in ascending order.

Selection - Sort (A)

for $i = 1$ to $n-1$ do

$\text{min} = i$

 for $j = i+1$ to n do

 // find the index of the i^{th} smallest element

 if $A[j] < A[\text{min}]$

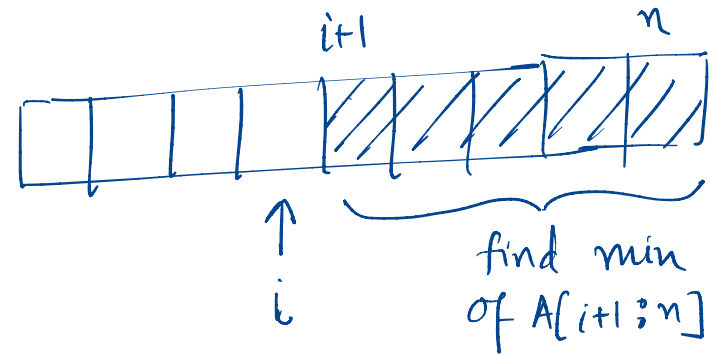
$\text{min} = j$

 end if

 end for

 swap $A[\text{min}]$ with $A[i]$

end for



→ Running time

$$T(n) = \sum_{i=1}^{n-1} (n-i) = n(n-1) - \sum_{i=1}^{n-1} i$$

$$= n(n-1) - \frac{n(n-1)}{2}$$

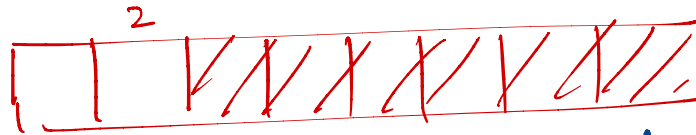
$$= \frac{n(n-1)}{2} = \frac{n^2 - n}{2} = \underline{\underline{\Theta(n^2)}}$$

$i = 1$



- a) find min. of $A[2:n] \Rightarrow$
b) swap with $A[1]$

$i = 2$



- a) find min of $A[3:n] \Rightarrow$ second min of A
b) swap with $A[2]$

⋮

$i = n-1$



— repeat

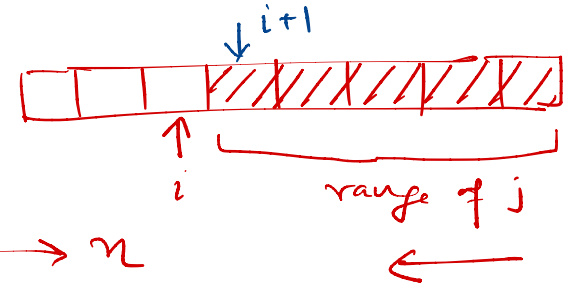
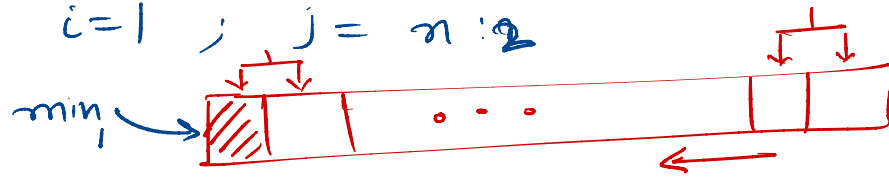
Bubble Sort

- Popular but inefficient sorting algorithm
- Works by repeatedly swapping adjacent elements that are out of order

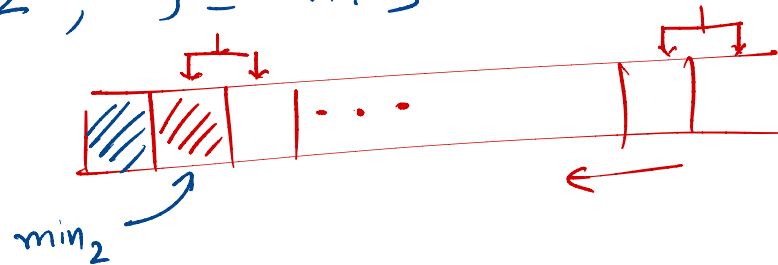
Bubble Sort (A)

```
for i = 1 to A.length - 1
  for j = A.length down to (i + 1)
    if A[j] < A[j - 1]
      exchange A[j] with A[j - 1]
    end if
  end for
end for
```

ex. Run :



$$i = 2 ; \quad j = n : 3$$

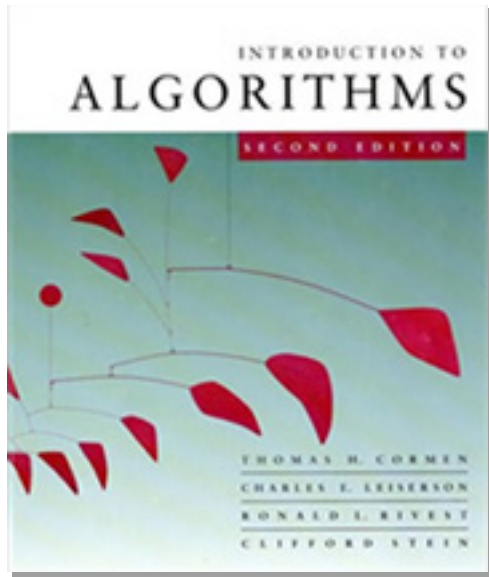


Loop invariant: At the start of each iteration, the subarray $A[1 : i-1]$ contains the $(i-1)$ smallest elements in sorted order.

$$\underline{T(n) = \Theta(n^2)}$$

Introduction to Algorithms

6.046J/18.401J

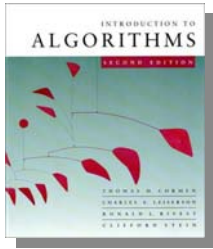


LECTURE 3

Divide and Conquer

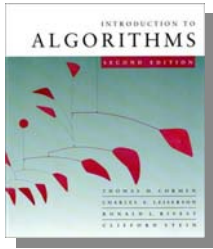
- Binary search
- Powering a number
- Fibonacci numbers
- Matrix multiplication
- Strassen's algorithm
- VLSI tree layout

Prof. Erik D. Demaine



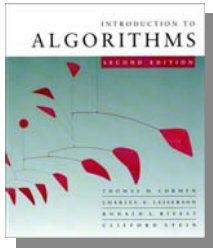
The divide-and-conquer design paradigm

1. *Divide* the problem (instance) into subproblems.
2. *Conquer* the subproblems by solving them recursively.
3. *Combine* subproblem solutions.



Merge sort

1. *Divide:* Trivial.
2. *Conquer:* Recursively sort 2 subarrays.
3. *Combine:* Linear-time merge.



Merge sort

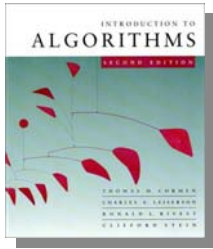
1. **Divide:** Trivial.
2. **Conquer:** Recursively sort 2 subarrays.
3. **Combine:** Linear-time merge.

$$T(n) = 2T(n/2) + \Theta(n)$$

subproblems

subproblem size

work dividing and combining



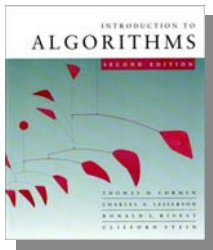
Master theorem (reprise)

$$T(n) = a T(n/b) + f(n)$$

CASE 1: $f(n) = O(n^{\log_b a - \epsilon})$, constant $\epsilon > 0$
 $T(n) = \Theta(n^{\log_b a})$.

CASE 2: $f(n) = \Theta(n^{\log_b a} \lg^k n)$, constant $k \geq 0$
 $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.

CASE 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$, constant $\epsilon > 0$,
and regularity condition
 $T(n) = \Theta(f(n))$.



Master theorem (reprise)

$$T(n) = a T(n/b) + f(n)$$

CASE 1: $f(n) = O(n^{\log_b a - \epsilon})$, constant $\epsilon > 0$
 $T(n) = \Theta(n^{\log_b a})$.

CASE 2: $f(n) = \Theta(n^{\log_b a} \lg^k n)$, constant $k \geq 0$
 $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.

CASE 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$, constant $\epsilon > 0$,
 and regularity condition

$$T(n) = \Theta(f(n)).$$

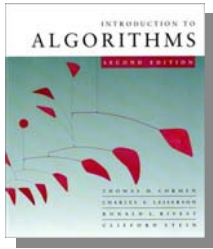
$$T(n) = 2T(n/2) + \Theta(n) \quad \text{with } f(n) \text{ written above and an arrow pointing to } \Theta(n)$$

Merge sort: $a = 2, b = 2$

$$n^{\log_b a} = \underline{n^{\log_2 2} = n}$$

CASE 2 ($k = 0$)

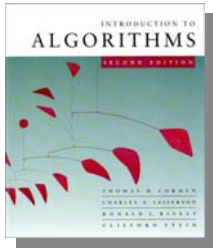
$$T(n) = \Theta(n \lg n).$$



Binary search

Find an element in a sorted array:

- 1. *Divide:*** Check middle element.
- 2. *Conquer:*** Recursively search **1** subarray.
- 3. *Combine:*** Trivial.



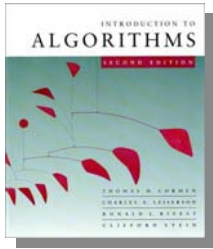
Binary search

Find an element in a sorted array:

- 1. *Divide:*** Check middle element.
- 2. *Conquer:*** Recursively search **1** subarray.
- 3. *Combine:*** Trivial.

Example: Find **9**

3 5 7 8 9 12 15



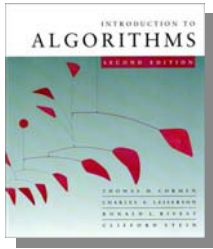
Binary search

Find an element in a sorted array:

- 1. Divide:** Check middle element.
- 2. Conquer:** Recursively search **1** subarray.
- 3. Combine:** Trivial.

Example: Find **9**





Binary search

Find an element in a sorted array:

- 1. Divide:** Check middle element.
- 2. Conquer:** Recursively search **1** subarray.
- 3. Combine:** Trivial.

Example: Find **9**

3

5

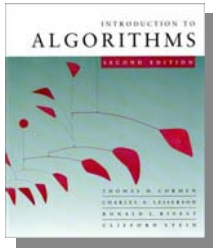
7

8

9

12

15



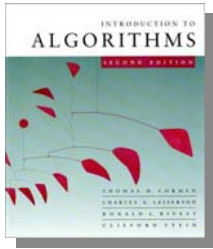
Binary search

Find an element in a sorted array:

- 1. Divide:** Check middle element.
- 2. Conquer:** Recursively search **1** subarray.
- 3. Combine:** Trivial.

Example: Find **9**

3 5 7 8 9 12 15



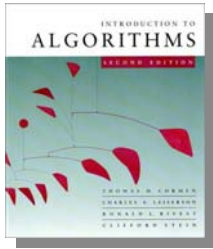
Binary search

Find an element in a sorted array:

- 1. *Divide:*** Check middle element.
- 2. *Conquer:*** Recursively search **1** subarray.
- 3. *Combine:*** Trivial.

Example: Find **9**

3 5 7 8 **9** 12 15



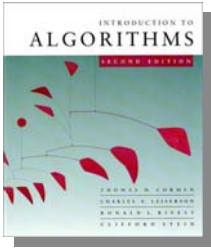
Binary search

Find an element in a sorted array:

- 1. *Divide:*** Check middle element.
- 2. *Conquer:*** Recursively search **1** subarray.
- 3. *Combine:*** Trivial.

Example: Find **9**

3 5 7 8 **9** 12 15



Recurrence for binary search

$$T(n) = 1T(n/2) + \Theta(1)$$

subproblems

subproblem size

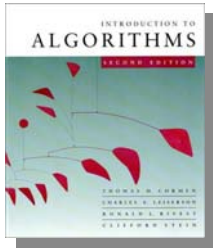
work dividing
and combining

$$T(n) = aT(n/b) + f(n) \quad : f(n) = \Theta(1)$$

2nd case of master th.

$$T(n) = \Theta(n^{\log_b a})$$
$$= \Theta(\lg n)$$

$a=1$
 $b=2$
 $k=0$
 $n^{\log_b a} = n^0 = 1$



Recurrence for binary search

$$T(n) = 1T(n/2) + \Theta(1)$$

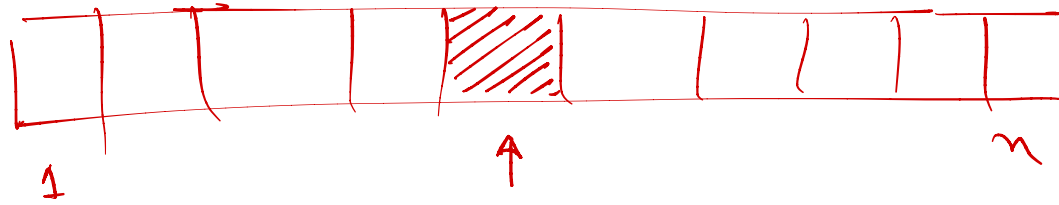
subproblems

subproblem size

work dividing
and combining

$$a = 1, b = 2 \quad \nearrow \theta(1) \quad f(n) = \theta(1) = c$$
$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1 \quad \text{CASE 2 } (k = 0)$$
$$T(n) = \Theta(\lg n).$$

Find v in A , given A is sorted :



recursive

(a)

$$mid = \left\lfloor \frac{n+1}{2} \right\rfloor$$

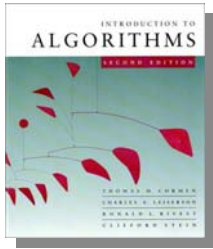
(b) Compare $A[mid]$ and v

→ Case 1: $A[mid] == v \rightarrow \text{return}$

Case 2: $A[mid] > v$

Case 3: $A[mid] < v$

↓
recurse



Powering a number

Problem: Compute a^n , where $n \in \mathbb{N}$.

Naive algorithm: $\Theta(n)$.

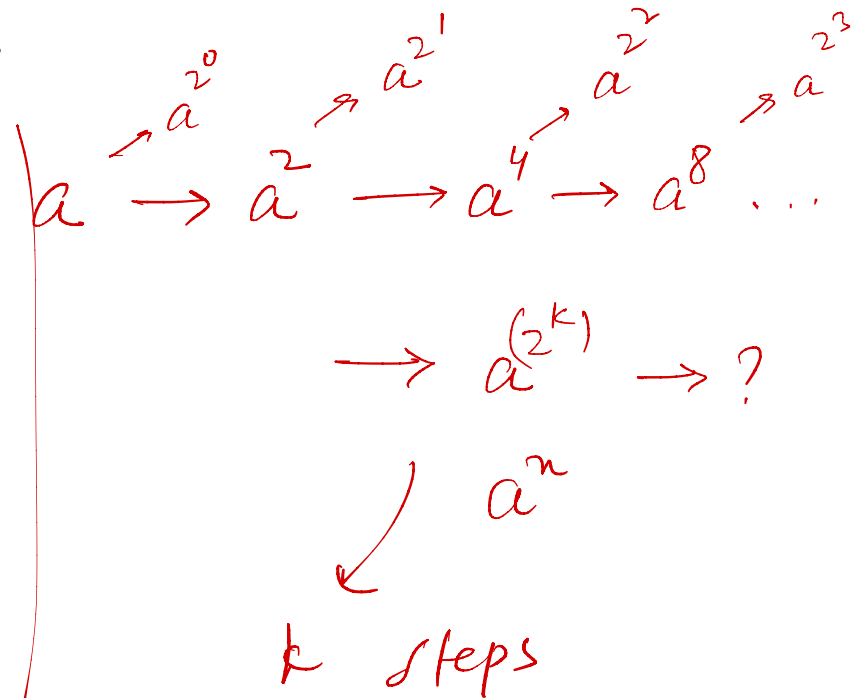
$n \rightarrow$ bin no.

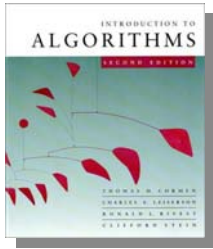


$$n = n_1 + n_2 + \dots + n_k$$

Powers of 2

$$a^n = a^{(n_1 + n_2 + \dots + n_k)} = \underbrace{(a^{n_1}) \cdot a^{n_2} \cdot \dots \cdot a^{n_k}}_{(k \text{ } n_k - (k-1))}$$





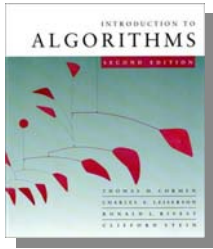
Powering a number

Problem: Compute a^n , where $n \in \mathbb{N}$.

Naive algorithm: $\Theta(n)$.

Divide-and-conquer algorithm:

$$a^n = \begin{cases} a^{n/2} \cdot a^{n/2} & \text{if } n \text{ is even;} \\ a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a & \text{if } n \text{ is odd.} \end{cases}$$



Powering a number

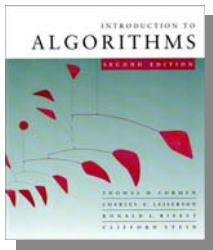
Problem: Compute a^n , where $n \in \mathbb{N}$.

Naive algorithm: $\Theta(n)$.

Divide-and-conquer algorithm:

$$a^n = \begin{cases} a^{n/2} \cdot a^{n/2} & \text{if } n \text{ is even;} \\ a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a & \text{if } n \text{ is odd.} \end{cases}$$

$$T(n) = T(n/2) + \Theta(1) \qquad T(n) = \Theta(\lg n) .$$



Fibonacci numbers

Recursive definition:

$$F_n = \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n \geq 2. \end{cases}$$

0 1 1 2 3 5 8 13 21 34 L

Recursion for naïve computation:

$$T(n) = T(n-1) + T(n-2) + \Theta(1)$$

$\rightarrow \Omega(\phi^n)$

Golden ratio $\phi = \left(\frac{1 + \sqrt{5}}{2} \right) \approx \frac{3.23}{2} = 1.61$

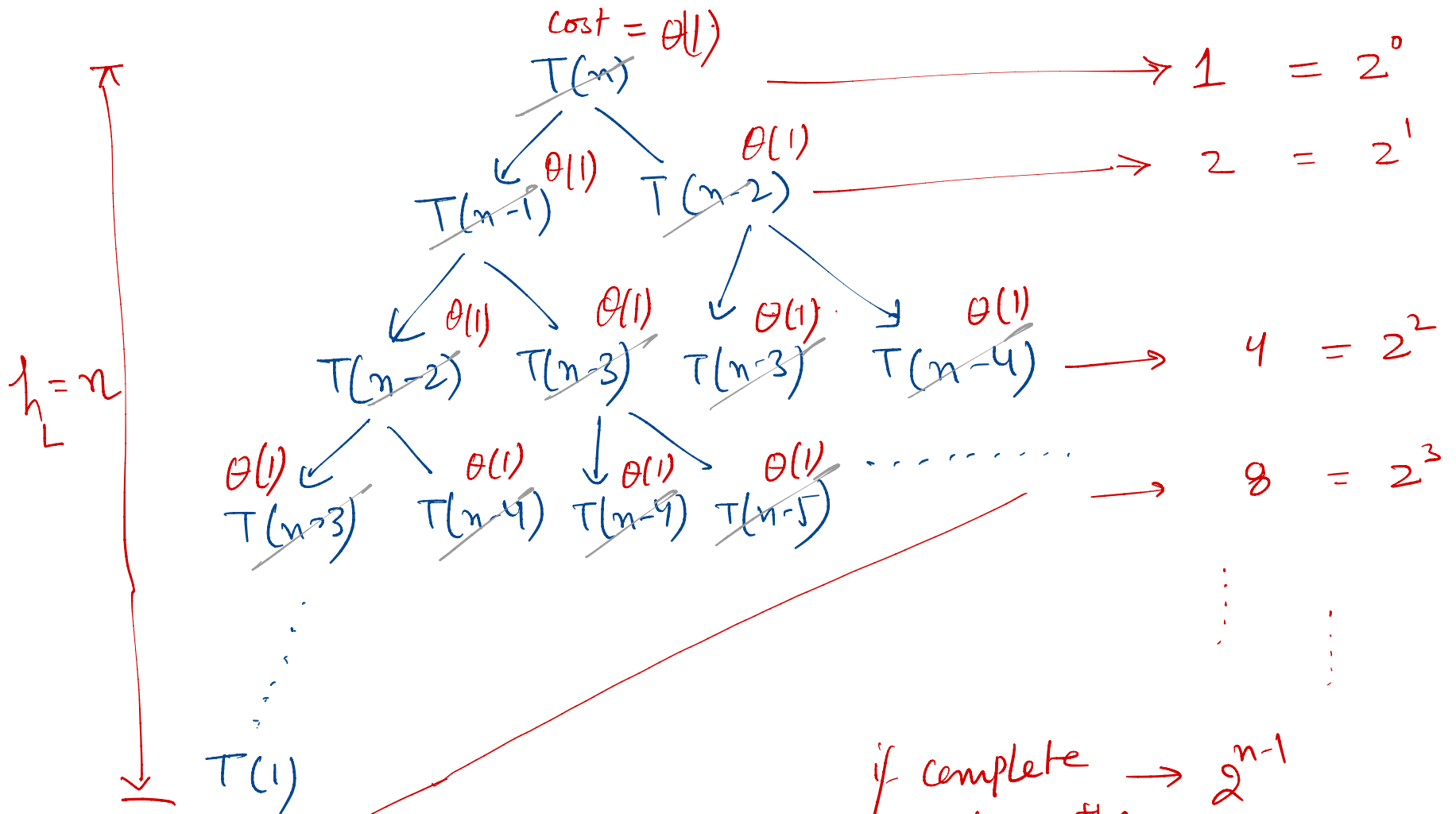
Two numbers a and b are in golden ratio,

If $a > b > 0$ and $\frac{a+b}{a} = \frac{a}{b} = \phi$

ϕ can also be written as a continued fraction

$$\phi = 1 + \frac{1}{1 + \frac{1}{1 + \frac{1}{1 + \frac{1}{1 + \dots}}}}$$

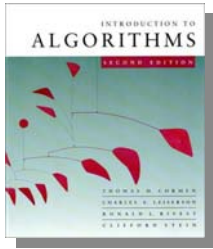
$$T(n) = T(n-1) + T(n-2) + \theta(1)$$



$$T(n) = O(2^n)$$

if complete tree, then 2^{n-1}

$$\sum_{i=1}^n 2^{i-1} = 2^n - 1$$



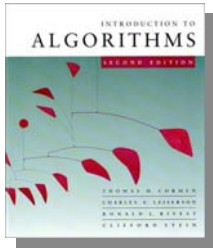
Fibonacci numbers

Recursive definition:

$$F_n = \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n \geq 2. \end{cases}$$

0 1 1 2 3 5 8 13 21 34 L

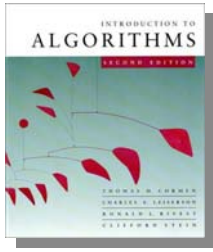
Naive recursive algorithm: $\Omega(\varphi^n)$
(exponential time), where $\varphi = (1 + \sqrt{5})/2$
is the *golden ratio*.



Computing Fibonacci numbers

Bottom-up:

- Compute $F_0, F_1, F_2, \dots, F_n$ in order, forming each number by summing the two previous.
- Running time: $\Theta(n)$.



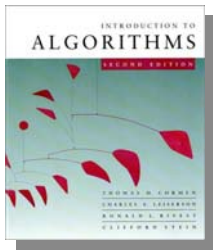
Computing Fibonacci numbers

Bottom-up:

- Compute $F_0, F_1, F_2, \dots, F_n$ in order, forming each number by summing the two previous.
 - Running time: $\Theta(n)$.
- Binet's formula

Naive recursive squaring:

- $F_n = \phi / \sqrt{5}$ rounded to the nearest integer.
- Recursive squaring: $\Theta(\lg n)$ time.
 - This method is unreliable, since floating-point arithmetic is prone to round-off errors.



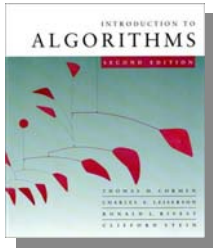
Recursive squaring

Theorem:

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n.$$

$$\Rightarrow \begin{bmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1}$$

$$\begin{bmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} F_n + F_{n-1} & F_n \\ F_{n-1} + F_{n-2} & F_{n-1} \end{bmatrix} = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}$$

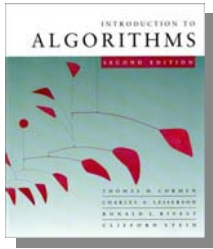


Recursive squaring

Theorem:

$$\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n.$$

Algorithm: Recursive squaring.
Time = $\Theta(\lg n)$.



Recursive squaring

Theorem:

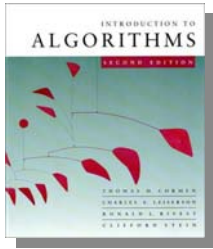
$$\begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n.$$

Algorithm: Recursive squaring.
Time = $\Theta(\lg n)$.

Proof of theorem. (Induction on n .)

Base ($n = 1$):

$$\begin{pmatrix} F_2 & F_1 \\ F_1 & F_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^1.$$

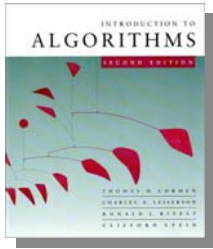


Recursive squaring

Inductive step ($n \geq 2$):

$$\begin{aligned}
 \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix} &= \begin{pmatrix} F_n & F_{n-1} & 1 & 1 \\ F_{n-1} & F_{n-2} & 1 & 0 \end{pmatrix} \cdot \\
 &= \begin{pmatrix} 1 & 1 & n-1 & 1 & 1 \\ 1 & 0 & & 1 & 0 \end{pmatrix} \cdot \\
 &= \begin{pmatrix} 1 & 1 & n \\ 1 & 0 & \end{pmatrix}
 \end{aligned}$$



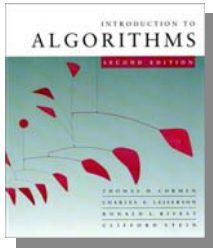


Matrix multiplication

Input: $A = [a_{ij}], B = [b_{ij}].$ } $i, j = 1, 2, \dots, n.$
Output: $C = [c_{ij}] = A \cdot B.$

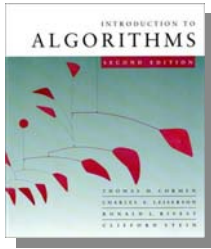
$$\begin{array}{cccc}
 c_{11} & c_{12} & \text{L} & c_{1n} \\
 c_{21} & c_{22} & \text{L} & c_{2n} \\
 \text{M} & \text{M} & \text{O} & \text{M} \\
 c_{n1} & c_{n2} & \text{L} & c_{nn}
 \end{array}
 =
 \begin{array}{cccc}
 a_{11} & a_{12} & \text{L} & a_{1n} \\
 a_{21} & a_{22} & \text{L} & a_{2n} \\
 \text{M} & \text{M} & \text{O} & \text{M} \\
 a_{n1} & a_{n2} & \text{L} & a_{nn}
 \end{array}
 \begin{array}{cccc}
 b_{11} & b_{12} & \text{L} & b_{1n} \\
 b_{21} & b_{22} & \text{L} & b_{2n} \\
 \text{M} & \text{M} & \text{O} & \text{M} \\
 b_{n1} & b_{n2} & \text{L} & b_{nn}
 \end{array}$$

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$



Standard algorithm

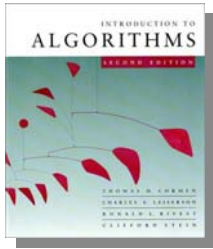
```
for  $i \leftarrow 1$  to  $n$ 
  do for  $j \leftarrow 1$  to  $n$ 
    do  $c_{ij} \leftarrow 0$ 
      for  $k \leftarrow 1$  to  $n$ 
        do  $c_{ij} \leftarrow c_{ij} + a_{ik} b_{kj}$ 
```



Standard algorithm

```
for  $i \leftarrow 1$  to  $n$ 
  do for  $j \leftarrow 1$  to  $n$ 
    do  $c_{ij} \leftarrow 0$ 
      for  $k \leftarrow 1$  to  $n$ 
        do  $c_{ij} \leftarrow c_{ij} + a_{ik} b_{kj}$ 
```

Running time = $\Theta(n^3)$



Divide-and-conquer algorithm

IDEA:

$n \times n$ matrix = 2 matrix of $(n/2) \times (n/2)$ submatrices:

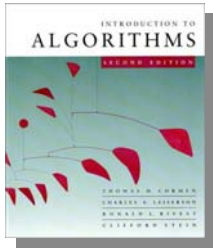
$$\begin{array}{c|c} r & s \\ \hline t & u \end{array} = \begin{array}{c|c} a & b \\ \hline c & d \end{array} \quad \begin{array}{c|c} e & f \\ \hline g & h \end{array}$$

$$C = A \quad B$$

$$\left. \begin{array}{l} r = ae + bg \\ s = af + bh \\ t = ce + dg \\ u = cf + dh \end{array} \right\}$$

8 mults of $(n/2) \times (n/2)$ submatrices

4 adds of $(n/2) \times (n/2)$ submatrices



Divide-and-conquer algorithm

IDEA:

$n \times n$ matrix = 2 matrix of $(n/2) \times (n/2)$ submatrices:

$$\begin{array}{c|c} r & s \\ \hline t & u \end{array} = \begin{array}{c|c} a & b \\ \hline c & d \end{array} \quad \begin{array}{c|c} e & f \\ \hline g & h \end{array}$$

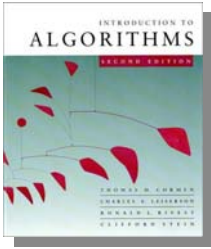
$$C = A \quad B$$

$$\left. \begin{array}{l} r = ae + bg \\ s = af + bh \\ t = ce + dh \\ u = cf + dg \end{array} \right\}$$

recursive

8 mults of $(n/2) \times (n/2)$ submatrices

4 adds of $(n/2) \times (n/2)$ submatrices



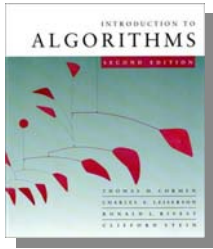
Analysis of D&C algorithm

$$T(n) = 8T(n/2) + \Theta(n^2)$$

submatrices

submatrix size

work adding
submatrices



Analysis of D&C algorithm

$$T(n) = 8T(n/2) + \Theta(n^2)$$

submatrices

submatrix size

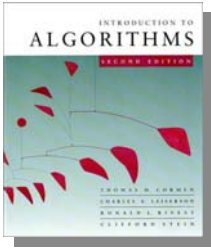
work adding
submatrices

$$a=8, b=2$$

$$n^{\log_b a} = n^{\log_2 8} = n^3$$

CASE 1

$$\underline{T(n) = \Theta(n^3).}$$



Analysis of D&C algorithm

$$T(n) = 8T(n/2) + \Theta(n^2)$$

submatrices

submatrix size

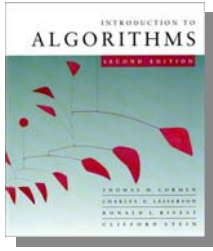
work adding
submatrices

$$n^{\log_b a} = n^{\log_2 8} = n^3$$

CASE 1

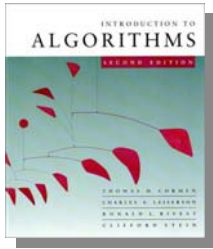
$$T(n) = \Theta(n^3).$$

No better than the ordinary algorithm.



Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.



Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \quad (f - h)$$

$$P_2 = (a + b) \quad h$$

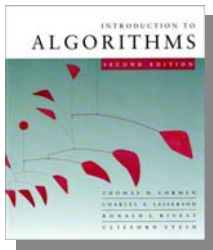
$$P_3 = (c + d) \quad e$$

$$P_4 = d \quad (g - e)$$

$$P_5 = (a + d) \quad (e + h)$$

$$P_6 = (b - d) \quad (g + h)$$

$$P_7 = (a - c) \quad (e + f)$$



Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \quad (f - h)$$

$$P_2 = (a + b) \quad h$$

$$P_3 = (c + d) \quad e$$

$$P_4 = d \quad (g - e)$$

$$P_5 = (a + d) \quad (e + h)$$

$$P_6 = (b - d) \quad (g + h)$$

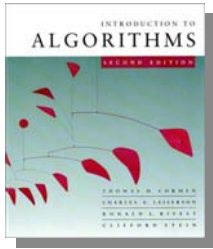
$$P_7 = (a - c) \quad (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$

$$s = P_1 + P_2$$

$$t = P_3 + P_4$$

$$u = P_5 + P_1 - P_3 - P_7$$



Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \quad (f - h)$$

$$P_2 = (a + b) \quad h$$

$$P_3 = (c + d) \quad e$$

$$P_4 = d \quad (g - e)$$

$$P_5 = (a + d) \quad (e + h)$$

$$P_6 = (b - d) \quad (g + h)$$

$$P_7 = (a - c) \quad (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$

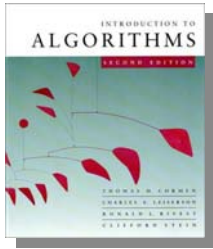
$$s = P_1 + P_2$$

$$t = P_3 + P_4$$

$$u = P_5 + P_1 - P_3 - P_7$$

7 mults, 18 adds/subs.

Note: No reliance on commutativity of mult!



Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \quad (f - h)$$

$$P_2 = (a + b) \quad h$$

$$P_3 = (c + d) \quad e$$

$$P_4 = d \quad (g - e)$$

$$P_5 = (a + d) \quad (e + h)$$

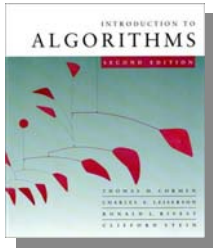
$$P_6 = (b - d) \quad (g + h)$$

$$P_7 = (a - c) \quad (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$

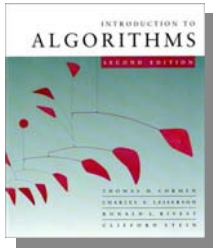
$$\begin{aligned} &= (a + d)(e + h) \\ &\quad + d(g - e) - (a + b)h \\ &\quad + (b - d)(g + h) \end{aligned}$$

$$\begin{aligned} &= ae + ah + de + dh \\ &\quad + dg - de - ah - bh \\ &\quad + bg + bh - dg - dh \\ &= ae + bg \end{aligned}$$



Strassen's algorithm

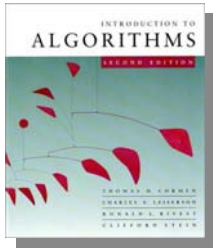
1. **Divide:** Partition A and B into $(n/2) \times (n/2)$ submatrices. Form terms to be multiplied using $+$ and $-$.
2. **Conquer:** Perform 7 multiplications of $(n/2) \times (n/2)$ submatrices recursively.
3. **Combine:** Form C using $+$ and $-$ on $(n/2) \times (n/2)$ submatrices.



Strassen's algorithm

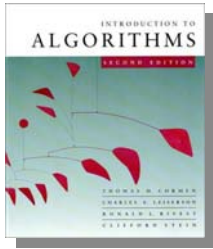
1. **Divide:** Partition A and B into $(n/2) \times (n/2)$ submatrices. Form terms to be multiplied using $+$ and $-$.
2. **Conquer:** Perform 7 multiplications of $(n/2) \times (n/2)$ submatrices recursively.
3. **Combine:** Form C using $+$ and $-$ on $(n/2) \times (n/2)$ submatrices.

$$T(n) = 7 T(n/2) + \Theta(n^2)$$



Analysis of Strassen

$$T(n) = 7 T(n/2) + \Theta(n^2)$$



Analysis of Strassen

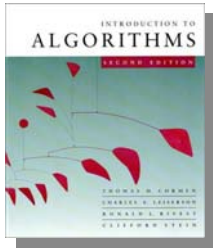
$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81}$$

CASE 1

$$T(n) = \Theta(n^{\lg 7}).$$

$$= \Theta(n^{2.81})$$

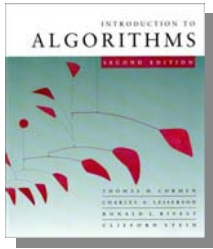


Analysis of Strassen

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81} \quad \text{CASE 1} \quad T(n) = \Theta(n^{\lg 7}).$$

The number 2.81 may not seem much smaller than 3, but because the difference is in the exponent, the impact on running time is significant. In fact, Strassen's algorithm beats the ordinary algorithm on today's machines for $n \geq 32$ or so.



Analysis of Strassen

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81} \quad \text{CASE 1} \quad T(n) = \Theta(n^{\lg 7}).$$

The number 2.81 may not seem much smaller than 3, but because the difference is in the exponent, the impact on running time is significant. In fact, Strassen's algorithm beats the ordinary algorithm on today's machines for $n \geq 32$ or so.

Best to date (of theoretical interest only): $\Theta(n^{2.3761})$.
 $\Theta(n^{2.371}) \leftarrow 2024$

Tower of Hanoi

- A puzzle containing 3 rods and a number of discs of various sizes (diameters) which can slide onto any rod.
- The puzzle begins with all discs stacked on one rod in order of decreasing size; smallest on top.

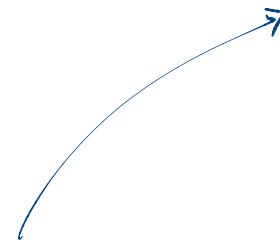
Objective : Move the entire stack to one of the other empty rods.

- Rules :**
1. Only 1 disc can be moved at a time
 2. discs can only be moved from/to top of a stack or on an empty rod.
 3. Only a smaller disc may be placed on top of an existing disc.

# discs	# steps
$n=1$	1
$n=2$	3
$n=3$	7
$n=4$	15
$\vdots$	
$n=k$	$2^k - 1$

Complexity

$$T(n) = 2^n - 1$$

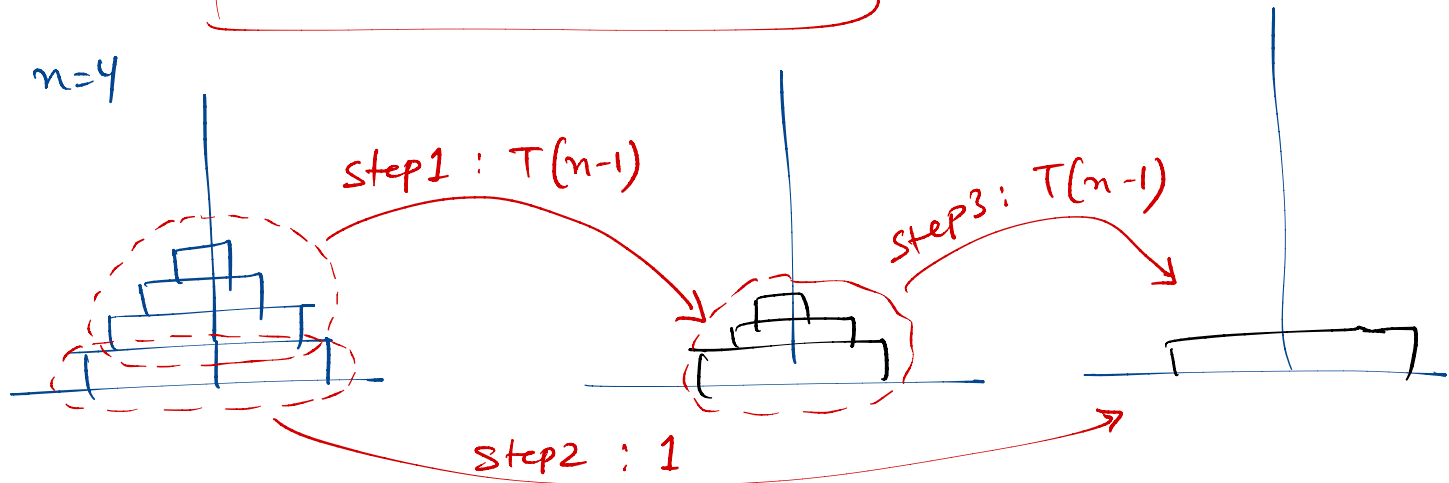


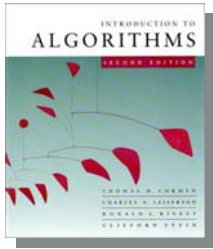
$$\Rightarrow T(n) = \Theta(2^n)$$

Recursion:

$$T(n) = 2T(n-1) + 1$$

lets take $n=4$





Conclusion

- Divide and conquer is just one of several powerful techniques for algorithm design.
- Divide-and-conquer algorithms can be analyzed using recurrences and the master method (so practice this math).
- The divide-and-conquer strategy often leads to efficient algorithms.